Testing Spring Boot Applications Demystified

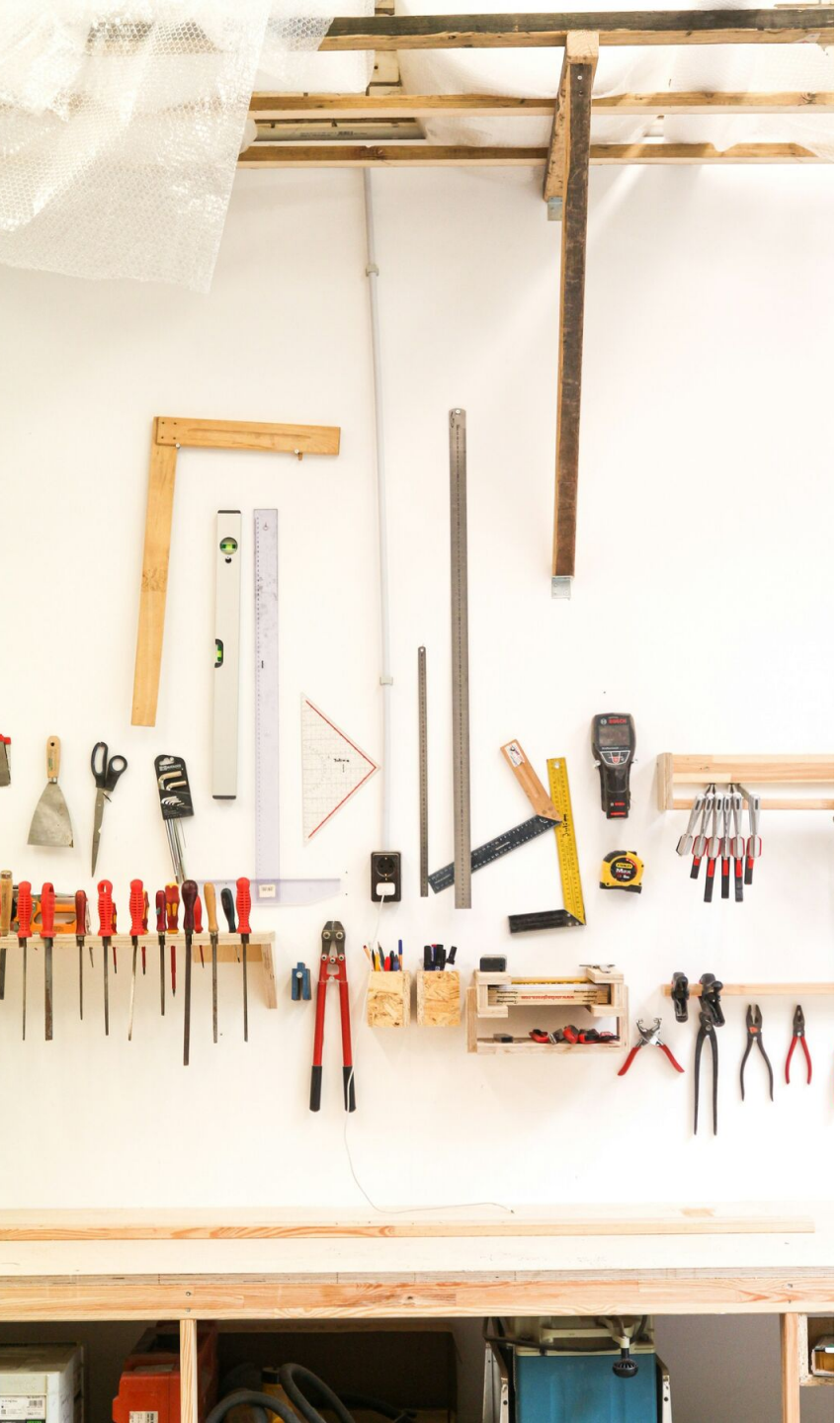
Lab 2

Digdir Workshop 02.03.2026

Philip Riecks - PragmaTech GmbH - @rieckpil

Discuss Exercises from Lab 1

- Exercise 1: Basic unit testing
- Exercise 2: JUnit Jupiter extension
- Exercise 3: Asserting with AssertJ



Lab 2

Sliced Testing - Introduction and Verifying the Web Layer

How Our Application Evolved

In Lab 1, we focused on **unit testing** plain Java classes in isolation:

- `BookService` tested with Mockito mocks
- No Spring context required
- Fast, simple, focused on business logic

Now, our application has grown and includes a **web layer**:

- REST controllers exposing HTTP endpoints
- Spring Security protecting certain routes
- JSON serialization/deserialization of request and response bodies

Imagine a Typical HTTP POST Controller Endpoint

```java
@RestController
@RequestMapping("/api/books")
public class BookController {

  private final BookService bookService;

  public BookController(BookService bookService) {
    this.bookService = bookService;
  }

  @PostMapping
  public ResponseEntity<Void> createBook(@Valid @RequestBody BookCreationRequest request, UriComponentsBuilder uriComponentsBuilder) {

    Long id = bookService.createBook(request);

    return ResponseEntity.created(
        uriComponentsBuilder.path("/api/books/{id}")
          .buildAndExpand(id)
          .toUri())
      .build();
  }
}
```

Unit Testing this Controller Endpoint

```java
@ExtendWith(MockitoExtension.class)
class BookControllerUnitTest {

  @Mock
  private BookService bookService;

  @InjectMocks
  private BookController bookController;

  @Test
  void shouldReturnCreatedResponseWithLocationWhenCreatingBook() {
    when(bookService.createBook(any(BookCreationRequest.class))).thenReturn(1L);

    BookCreationRequest creationRequest = new BookCreationRequest(
      "123-1234567890",
      "Test Book",
      "Test Author",
      LocalDate.of(2020, 1, 1)
    );

    ResponseEntity<Void> response = bookController
      .createBook(creationRequest, UriComponentsBuilder.newInstance().scheme("http").host("localhost"));

    assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
  }
}
```

Unit Testing Has Limits

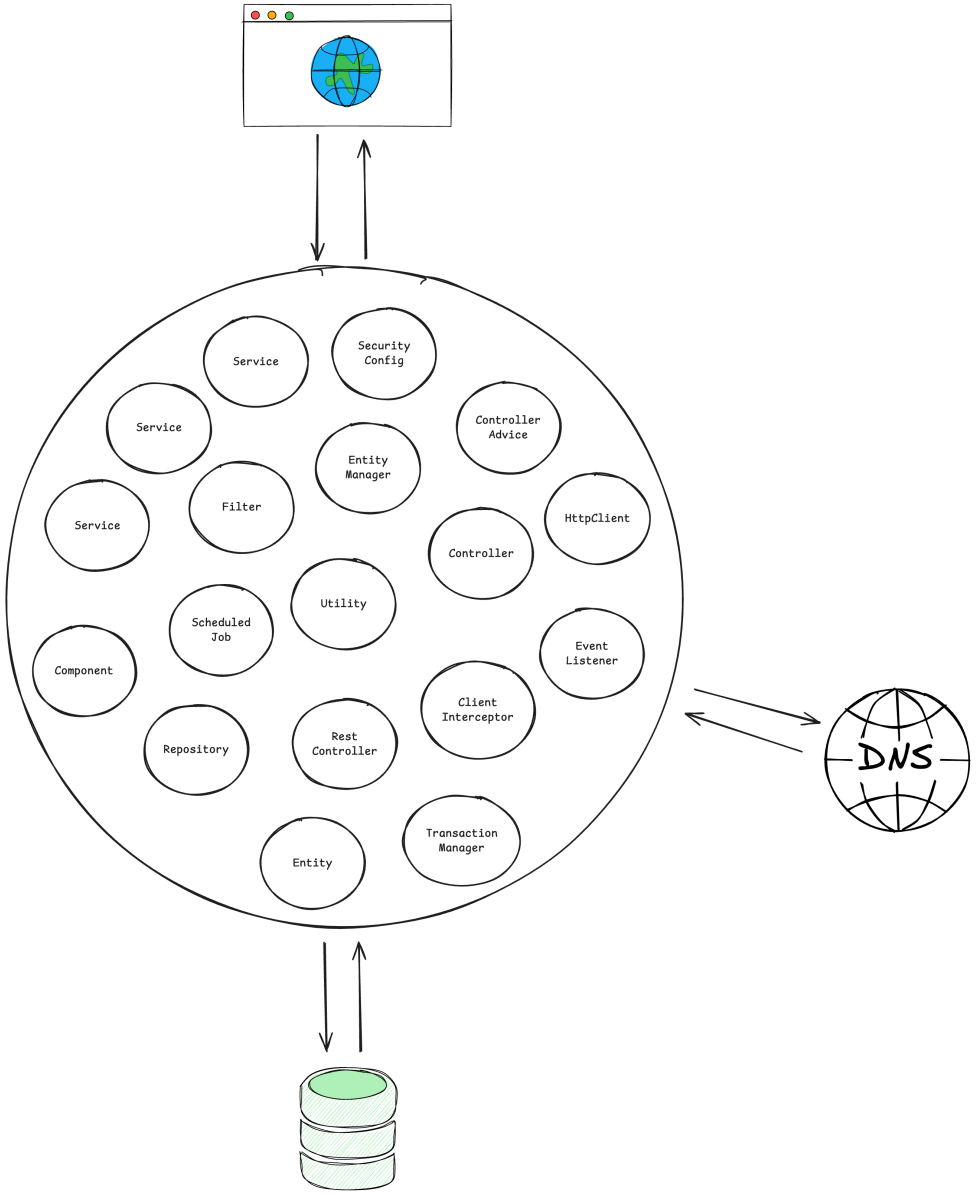
With (only) unit testing, we get limited confidence when it comes to:

- **Request Mapping**: Does `/api/books/{isbn}` actually resolve to your desired method?
- **Validation**: Will incomplete request bodies result in a 400 Bad Request or return an accidental 200?
- **Serialization**: Are your JSON objects serialized and deserialized correctly?
- **Headers**: Are you setting Content-Type or custom headers correctly?
- **Security**: Are your Spring Security configuration and other authorization checks enforced?

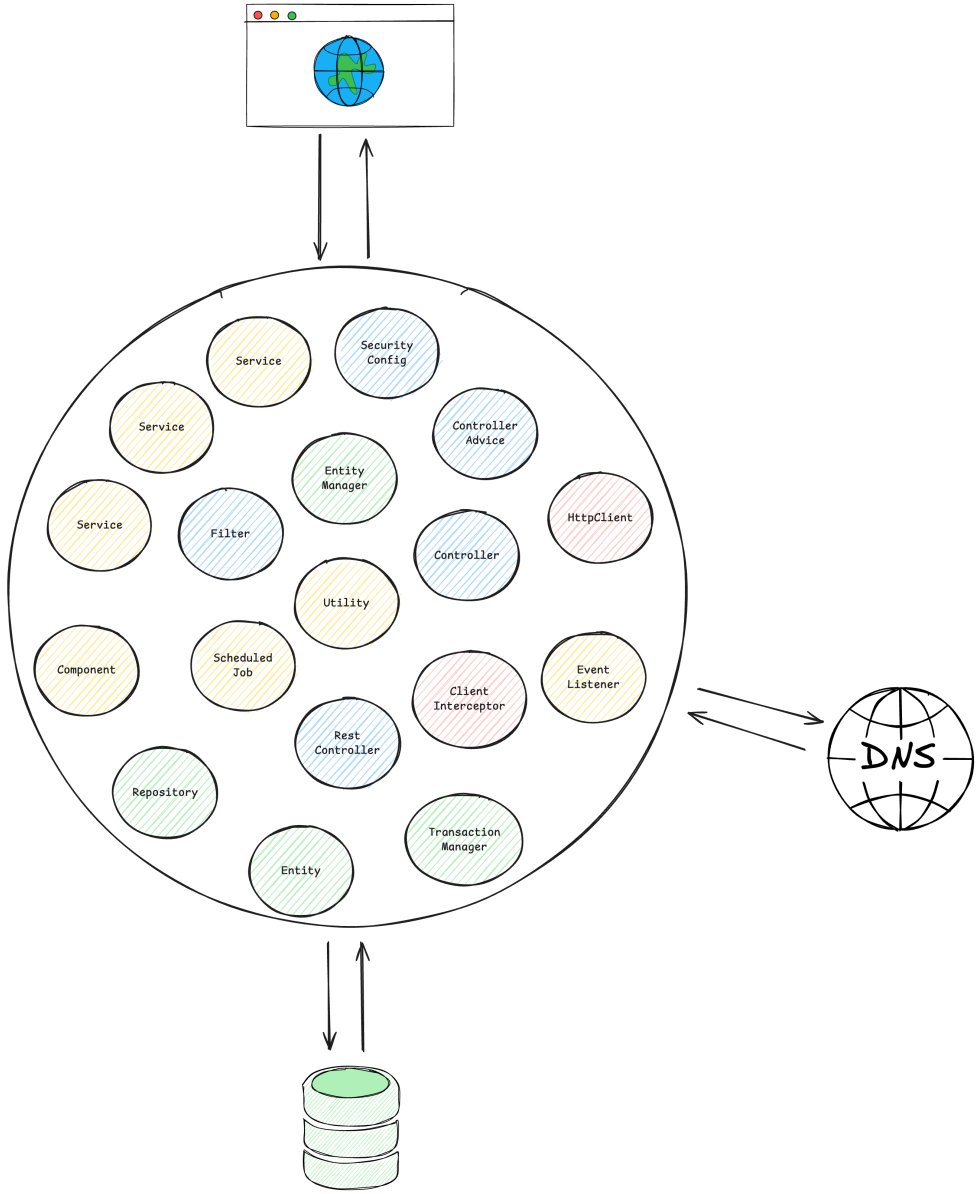
Finding a Better Alternative

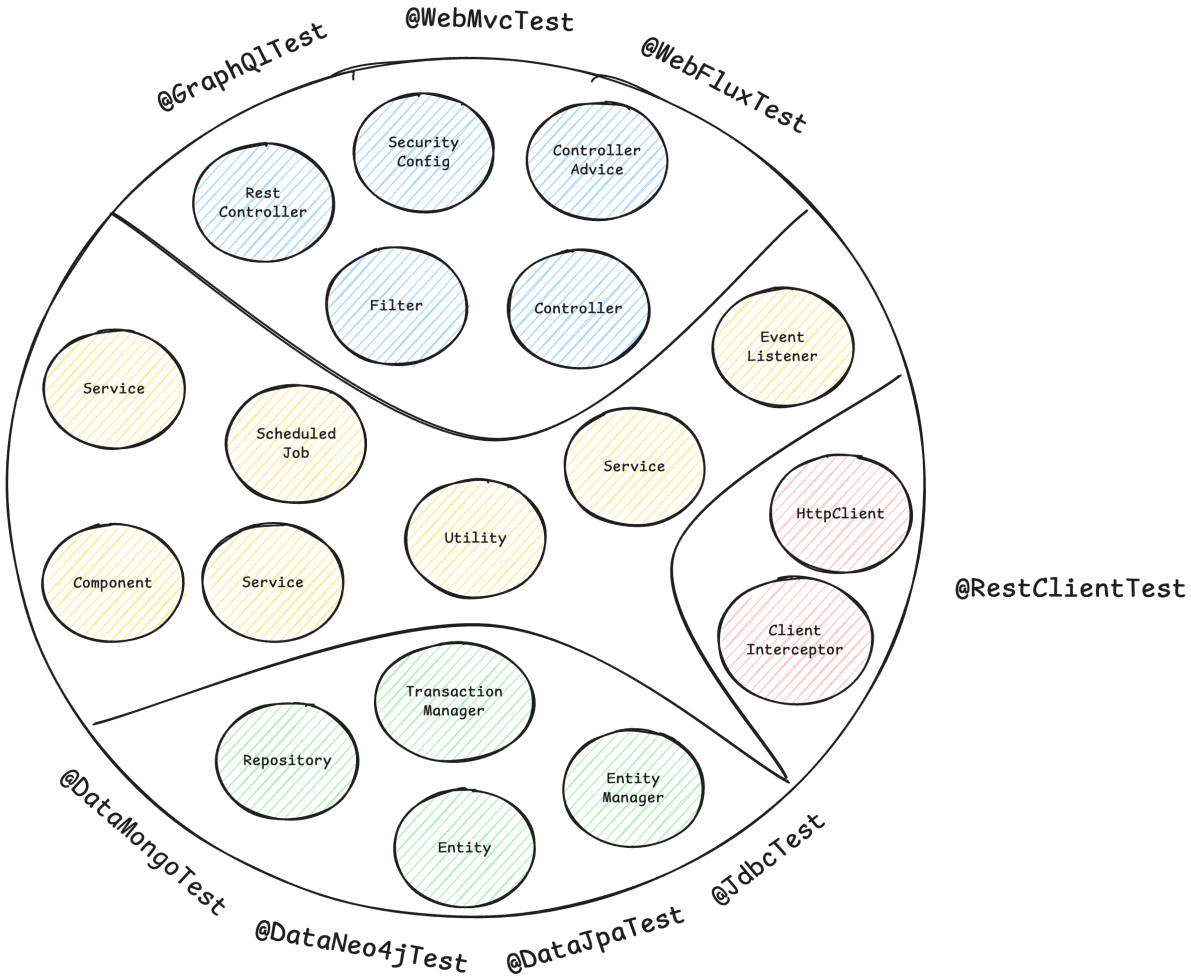
Sliced Testing

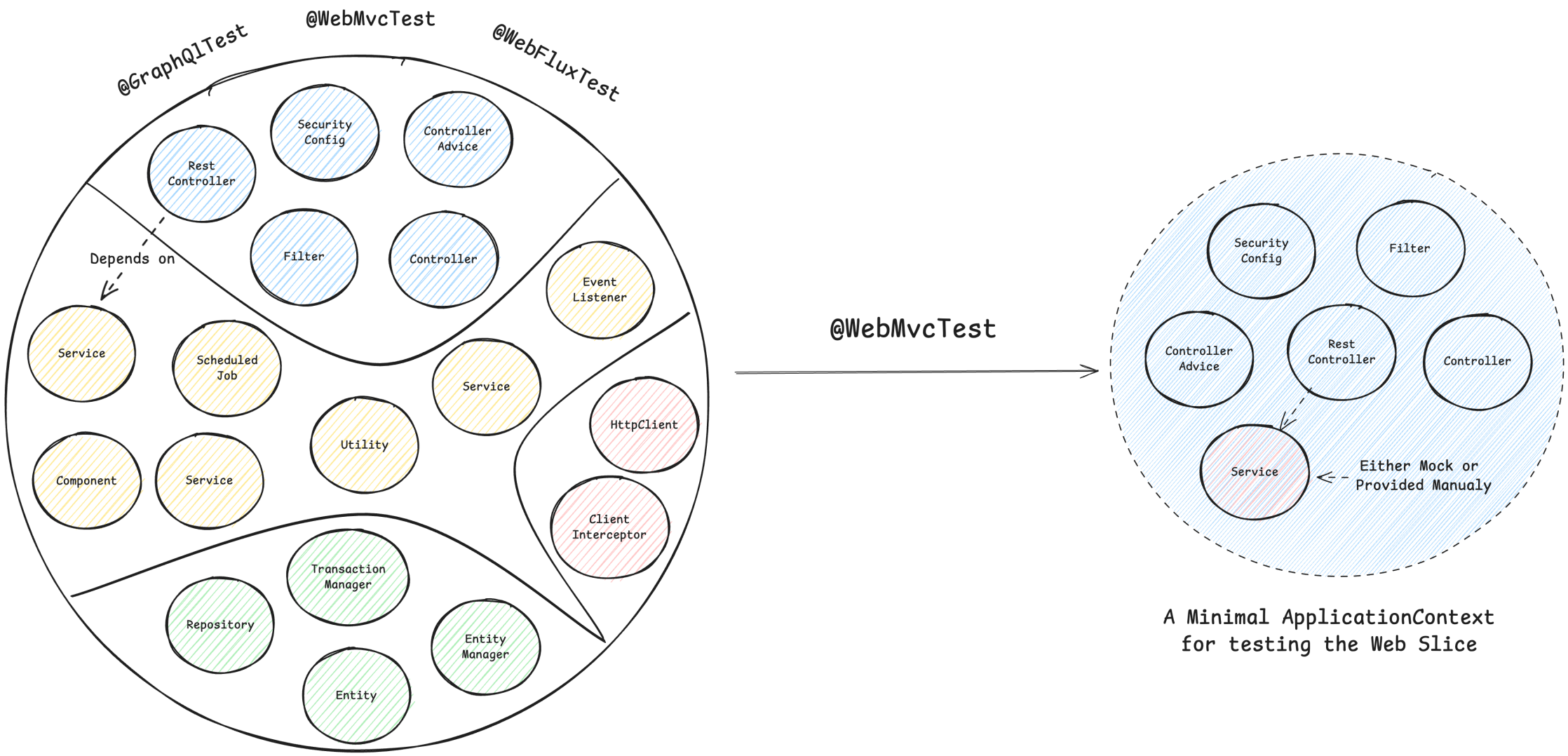
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext




@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Event Listener
Service
Scheduled Job
Service
Utility
HttpClient
Component
Service
Client Interceptor
@RestClientTest
Transaction Manager
Repository
Entity Manager
Entity
@DataMongoTest
@DataNeo4jTest
@DataJpaTest
@JdbcTest


@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Depends on
Event Listener
Service
Scheduled Job
Service
Utility
HttpClient
Component
Service
Client Interceptor
Transaction Manager
Repository
Entity Manager
Entity
@WebMvcTest
Security Config
Filter
Controller Advice
Rest Controller
Controller
Service
Either Mock or Provided Manualy
A Minimal ApplicationContext for testing the Web Slice

Spring Boot Test Slice Example: `@WebMvcTest`

```java
@WebMvcTest(BookController.class)
@Import(SecurityConfig.class)
class BookController {

  @Autowired
  private MockMvc mockMvc;

  @MockitoBean
  private BookService bookService;

  @Test
  @WithMockUser
  void shouldReturnLocationOfNewlyBook() throws Exception {
    // ...
  }
}
```

What Is `MockMvc`?

- A **mocked servlet environment** provided by Spring Test and auto-configured by Spring Boot with `@WebMvcTest`
- Simulates HTTP requests **without starting an actual server** (no real Tomcat)
- Processes the full Spring MVC pipeline: routing, filters, serialization, exception handling
- Allows testing controllers with real HTTP semantics (status codes, headers, body)

```
this.mockMvc
  .perform(get("/api/books/1234")
  .accept(MediaType.APPLICATION_JSON))
  .andExpect(status().isOk())
  .andExpect(jsonPath("$.title").value("Spring Boot Testing"));
```

MockMvc: What Gets Tested?

Aspect	Unit Test	MockMvc
Business logic	✓	✓
Request mapping	✗	✓
JSON serialization	✗	✓
Validation (`@Valid`)	✗	✓
Exception handling	✗	✓
Security filters	✗	✓
Content negotiation	✗	✓

Slicing Example in Action `@WebMvcTest`

```
@WebMvcTest(BookController.class)
@Import(SecurityConfig.class)
class BookControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @MockitoBean
  private BookService bookService;

}
```

What `@WebMvcTest` Loads (and What It Doesn't)

Included in the sliced context:

- `@Controller`, `@RestController`, `@ControllerAdvice`
- `@JsonComponent`, `Converter`, `Filter`
- `WebMvcConfigurer`, `HandlerMethodArgumentResolver`

Excluded from the sliced context:

- `@Service`, `@Repository`, `@Component`
- `DataSource`, `EntityManager`, JPA repositories
- Any non-web Spring beans

Use `@MockitoBean` to provide mocks for excluded dependencies.

Strategies for Adding Missing Beans to a Slice

Strategy	When to Use
`@MockitoBean`	Replace a dependency with a Mockito mock — most common approach
`@MockitoSpyBean`	Wrap the real bean to verify interactions while keeping real behavior
`@TestConfiguration` + `@Bean`	Provide a custom bean definition scoped to your test
`@Primary` in a test config	Override an existing bean with a test-specific implementation

Adding Missing Beans: Code Examples

```java
// 1. Mock — no real behavior
@MockitoBean
private BookService bookService;

// 2. Spy — real behavior + verification
@MockitoSpyBean
private BookService bookService;

// 3. TestConfiguration — custom bean for the test
@TestConfiguration
static class TestConfig {
  @Bean
  public BookService bookService() {
    return new BookService(new InMemoryBookRepository());
  }
}

public class InMemoryBookRepository implements BookRepository {
  private final Map<String, Book> store = new HashMap<>();
}
```

Simulating Authenticated Users

Spring Security Test provides annotations and request post-processors:

```
@Test
@WithMockUser(roles = "ADMIN")
void shouldAllowAdminToDeleteBook() throws Exception {
  this.mockMvc
    .perform(delete("/api/books/1234"))
    .andExpect(status().isNoContent());
}

@Test
@WithMockUser(roles = "USER")
void shouldForbidRegularUserFromDeletingBook() throws Exception {
  this.mockMvc
    .perform(delete("/api/books/1234"))
    .andExpect(status().isForbidden());
}
```

Spring Security Test Override Options

Approach	Use Case
`@WithMockUser`	Quick mock with roles/authorities
`@WithMockUser(username, roles)`	Customized mock principal
`SecurityMockMvcRequestPostProcessors`	Test with specific authentication (e.g. basic auth)
No annotation	Verify unauthenticated access is rejected

Spring Security Test: Under the Hood

- `SecurityContextHolder` stores the authentication per thread (`ThreadLocal`)
- Before the test runs, the mock authentication is placed on the current thread
- After the test, the context is cleaned up automatically

```
// What @WithMockUser and SecurityMockMvcRequestPostProcessors do internally
// see TestSecurityContextHolder
SecurityContext context = SecurityContextHolder.createEmptyContext();

context.setAuthentication(
  new UsernamePasswordAuthenticationToken("user", "password",
    List.of(new SimpleGrantedAuthority("ROLE_USER")))
);

SecurityContextHolder.setContext(context);
```

Common Test Slices

- `@WebMvcTest` - Controller layer
- `@DataJpaTest` - Repository layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- `@WebFluxTest` - WebFlux controller testing
- `@JdbcTest` - JDBC testing

@DataCouchbaseTest

@DataLdapTest

@RestClientTest

@JsonTest

@JooqTest

@WebFluxTest

@DataRedisTest

@WebMvcTest

@DataMongoTest

@DataCassandraTest

@GraphQlTest

@DataNeo4jTest

@WriteYourOwn*

@DataJpaTest

@SqsTest

@JdbcTest

@DataElasticsearchTest

Sliced Testing Spring Boot Applications 101

- **Core Concept**: Test a specific "slice" or layer of your application by loading a minimal, relevant part of the Spring `ApplicationContext`.
- **Confidence Gained**: Helps validate parts of your application where pure unit testing is insufficient, like the web, messaging, or data layer.
- **Prominent Examples:** Web layer (`@WebMvcTest`) and database layer (`@DataJpaTest` -> next lab)
- **Pitfalls**: Requires careful configuration to ensure only the necessary slice of the context is loaded.
- **Tools**: JUnit, Mockito, Spring Test, Spring Boot

Time For Some Exercises

Lab 2

- Work with the known code repository on the same git branch
- Navigate to the `labs/lab-2` folder in the repository and complete the tasks as described in the `README` file of that folder
- Time boxed until the end of the lunch break (13:30)